

Programming Assignment 2

Let Go of Them (Redundant Load Elimination)

CS6004: Code Optimization for Object-Oriented Languages

Spring 2026

Due Date: February 15th, 2026

1 Introduction and Motivation

Object-oriented languages such as Java frequently perform field accesses within loops, conditional branches, and method invocations. In many programs, the same object field is loaded repeatedly even though its value remains unchanged between successive accesses. These *redundant field loads* lead to unnecessary memory operations and degrade runtime performance by cluttering the intermediate representation with repetitive instructions. From the perspective of static analysis, redundant loads also reduce analysis precision by introducing extra dependencies and statements that do not contribute new information. Eliminating such redundancies is therefore beneficial both for optimization and for improving the clarity and effectiveness of program analyses. This assignment focuses on improving field access precision by identifying and listing redundant field loads and its possible replacement using static analysis.

2 Detailed Specification

At every program point, we want to know: "*Which object fields have already been loaded, and are still valid so that a later load is redundant?*" So when we see a statement $x = o.f$ we can decide it is redundant if:

- the same (o, f) has been loaded into x earlier on all paths, and
- there has been no write to $o.f$ since that load.

The objective of this assignment is to implement and perform:

1. **Intraprocedural points-to information:** You must compute **flow-** and **field-sensitive** intraprocedural points-to information for each method.
2. Since the analysis is intraprocedural, you may assume conservative results at call sites.
3. You can assume single-threaded execution; no parallel code execution occurs.
4. **What to print?** Using the generated points-to information, identify the source code line numbers where one or more load operations are redundant and can be replaced by an existing variable. The results should be printed for each class, organized by method and sorted in lexicographic order, printing the class and method name only if at least one redundant load exists.
 - You can use `getJavaSourceLineNumber()` method to obtain the line number of a `Unit` corresponding to the source program.
 - For a given line, if multiple outputs exist, print each output separately.
 - You can ignore the results for constructors and libraries while printing the final result.
 - You should print method name/class name only if there is at least one redundant load.
5. We will provide a "starter_PA2.tar.gz" file that you can expand upon for the assignment.

3 Sample Input Programs

Testcase 1:

```
1 class Node {
2     Node f1;
3     Node f2;
4     Node g;
5     Node() {}
6 }
7
8 public class Test {
9     public static void main(String[] args) {
10         Node a = new Node(); // 010
11         a.f1 = new Node(); // 011
12         Node b = new Node(); // 012
13         b.f1 = new Node(); // 013
14         a.f2 = new Node(); // 014
15         Node c = a.f1;
16         a.f2 = a.f1; // Redundant
17         b.f1 = a.f2; // Redundant
18     }
19 }
```

Testcase 2:

```
1 public class Test {
2     int f1;
3     public static void main(String[] args) {
4         Test a = new Test(); // 04
5         a.f1 = 10;
6         int b = a.f1;
7         int c = a.f1; // Redundant
8         a.foo();
9         int d = a.f1;
10    }
11    void foo() {
12        Test o1 = new Test(); // 012
13        int x;
14        o1.f1 = 20;
15        Test o2 = o1;
16        x = o1.f1;
17        int y = o2.f1; // Redundant
18    }
19 }
```

Generated Jimple:

Testcase 1:

```
public static void main(java.lang.String[])
{
    Node $r0, $r1, $r2, $r3, $r4, r5, $r6, $r7;
    java.lang.String[] r8;
    r8 := @parameter0: java.lang.String[];
    $r0 = new Node;
    specialinvoke $r0.<Node: void <init>()>();
    $r1 = new Node;
    specialinvoke $r1.<Node: void <init>()>();
    $r0.<Node: Node f1> = $r1;
    $r2 = new Node;
    specialinvoke $r2.<Node: void <init>()>();
    $r3 = new Node;
    specialinvoke $r3.<Node: void <init>()>();
    $r2.<Node: Node f1> = $r3;
    $r4 = new Node;
    specialinvoke $r4.<Node: void <init>()>();
    $r0.<Node: Node f2> = $r4;
    r5 = $r0.<Node: Node f1>;
    $r6 = $r0.<Node: Node f1>;
    $r0.<Node: Node f2> = $r6;
    $r7 = $r0.<Node: Node f2>;
    $r2.<Node: Node f1> = $r7;
    return;
}
```

Testcase 2:

```
public static void main(java.lang.String[])
{
    Test $r0;
    int i0, i1, i2;
    java.lang.String[] r1;
    r1 := @parameter0: java.lang.String[];
    $r0 = new Test;
    specialinvoke $r0.<Test: void <init>()>();
    $r0.<Test: int f1> = 10;
    i0 = $r0.<Test: int f1>;
    i1 = $r0.<Test: int f1>;
    virtualinvoke $r0.<Test: void foo()>();
    i2 = $r0.<Test: int f1>;
    return;
}

void foo(boolean)
{
    Test $r0, r1;
    int i0, i1;
    boolean z0;
    r1 := @this: Test;
    z0 := @parameter0: boolean;
    $r0 = new Test;
    specialinvoke $r0.<Test: void <init>()>();
    $r0.<Test: int f1> = 20;
    i0 = $r0.<Test: int f1>;
    i1 = $r0.<Test: int f1>;
    return;
}
```

Expected Output:

Output

Output

```
Test:main
16:$r0.<Node:Node f1> r5
17:$r0.<Node:Node f2> r5
```

```
Test:foo
17:$r0.<Test:int f1> i0
Test:main
7:$r0.<Test:int f1> i0
```

Output Format: `SourceCode_LineNumber:LoadStatement_in_jimple jimp_var_name`

4 Running the Starter Code

1. Compile Testcase:

```
javac Test.java
```

2. Compile Analysis:

```
javac -cp .:soot-4.6.0-jar-with-dependencies.jar PA2.java
```

3. Run: (Note - Pass Testcase folder name [Test1 or Test2] as argument)

```
java -cp .:soot-4.6.0-jar-with-dependencies.jar PA2 Test1
```

Marks for each testcase will be awarded if the printed output matches the expected output.

5 Submission Instructions